操作系统原理 lab3 实验报告

实验课程:操作系统原理

实验名称:**从实模式到保护模式**

专业名称:计算机科学与技术

学生姓名:梁力航

学生学号:23336128

实验地点:东实验楼 B201

1. 实验要求

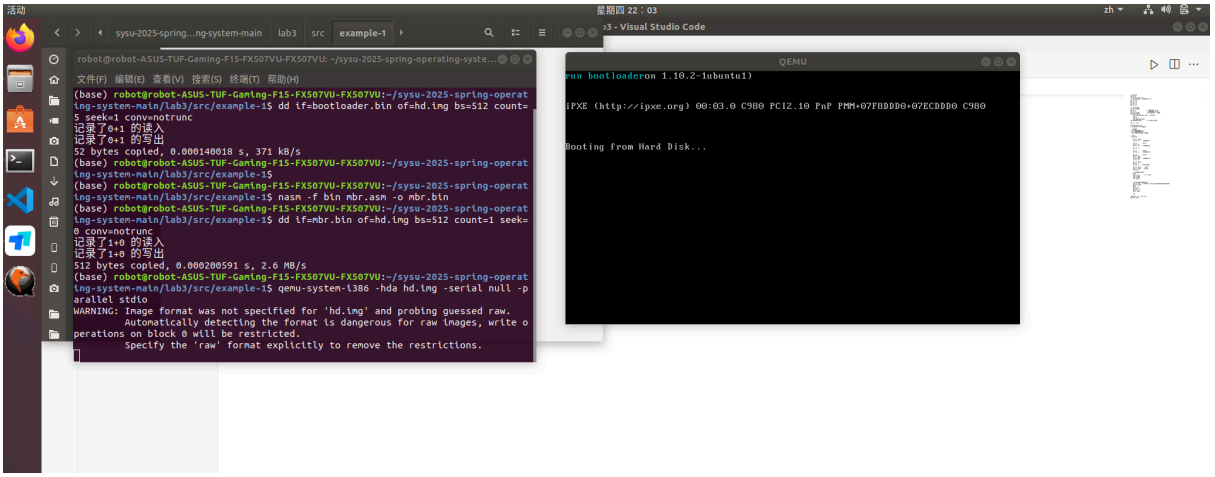
1. 阅读实验教程，掌握基础知识
2. 复现Example 1
3. 将LBA28读取硬盘的方式换成CHS读取，同时给出逻辑扇区号向CHS的转换公式
4. 复现Example 2，使用gdb或其他debug工具在进入保护模式的4个重要步骤上设置断点，并结合代码、寄存器的内容等来分析这4个步骤

2. 实验过程

任务一

复现example1

```
nasm -f bin bootloader.asm -o bootloader.bin
dd if=bootloader.bin of=hd.img bs=512 count=5 seek=1 conv=notrunc
nasm -f bin mbr.asm -o mbr.bin
dd if=mbr.bin of=hd.img bs=512 count=1 seek=0 conv=notrunc
qemu-system-i386 -hda hd.img -serial null -parallel stdio
```



将LBA28读取硬盘的方式换成CHS读取

1. CHS参数转换逻辑

```
; LBA转CHS公式实现
mov dx, 0
mov cx, 63     ; 每磁道63扇区
div cx         ; AX=柱面/磁头商，DX=扇区余数
inc dx         ; 扇区号修正(+1)
mov cl, dl     ; CL低6位存储扇区号
...
mov cx, 18     ; 每柱面18磁头
div cx         ; 分解柱面与磁头号
```

- **转换公式**：

`柱面号 = LBA / (磁头数×扇区数)`

`磁头号 = (LBA / 扇区数) % 磁头数`

`扇区号 = LBA % 扇区数 + 1`

- **参数存储规则**：

CH寄存器存储柱面号低8位，CL高2位存储柱面号高位，DH存储磁头号

2. BIOS中断调用

```
mov dl, 0x80      ; 主硬盘驱动器号
mov ax, 0x0201    ; AH=02h(读扇区), AL=01h(读1个扇区)
mov bx, 0x8000    ; 缓冲区地址ES:BX=0000:8000
```

```
int 0x13        ; 触发磁盘中断
jc .error       ; 错误处理分支
```

- **寄存器参数映射**：

- CH=柱面号低8位
- CL=扇区号(低6位) + 柱面高2位
- DH=磁头号
- DL=驱动器号(0x80为第一硬盘)
- **错误处理机制**：通过进位标志(CF)判断操作结果，错误码存储在AH寄存器

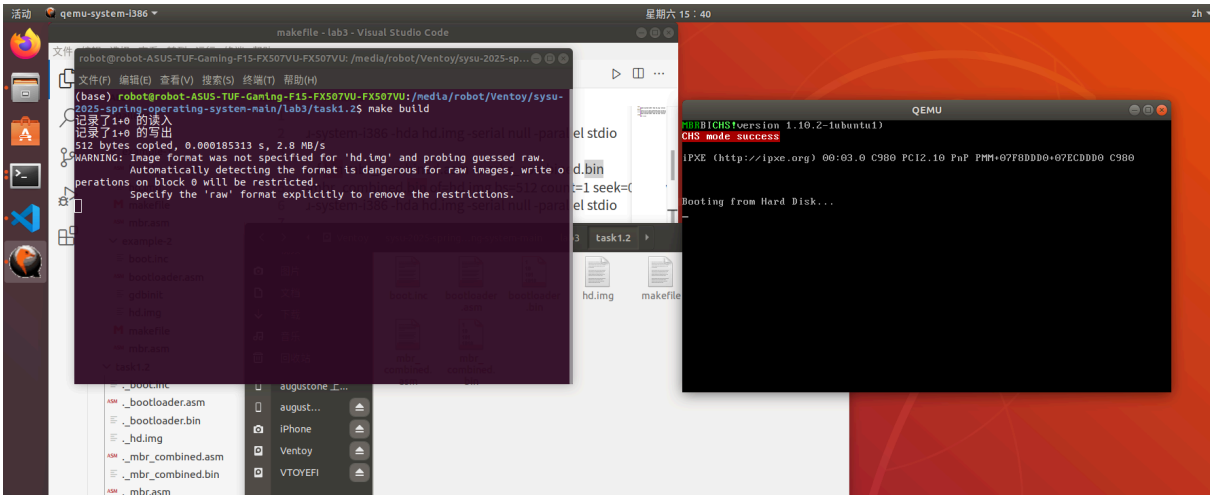
3. 显存直接操作

```
mov ax, 0xb800   ; 文本模式显存段地址
mov gs, ax       ; 通过GS段覆盖访问
mov ah, 0x0A     ; 字符属性(亮绿色背景)
mov al, 'C'      ; ASCII字符
mov [gs:10], ax  ; 写入显存偏移地址
```

- **显示原理**：每个字符占用2字节(ASCII+属性)，通过段寄存器GS实现快速访问
- **颜色编码**：

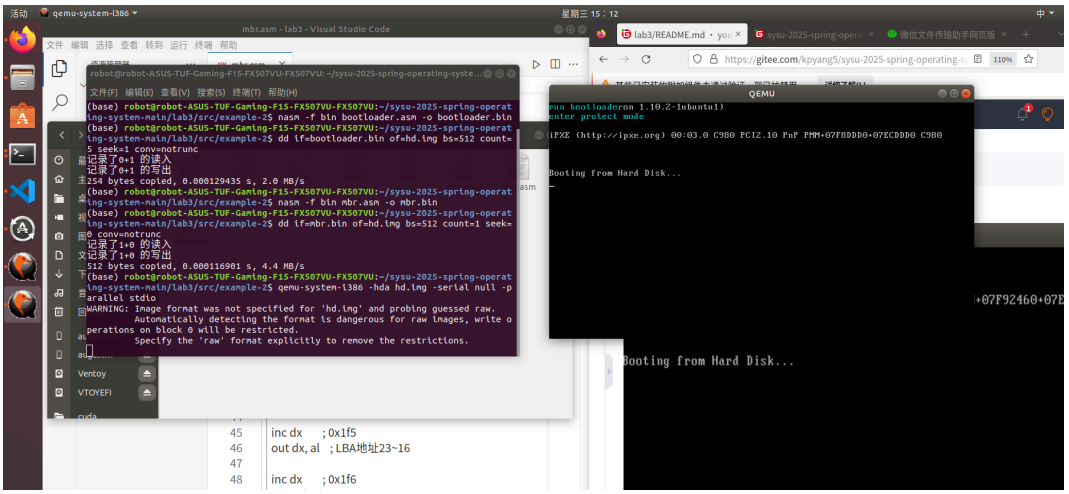
- 0x02=绿色提示符
- 0x4F=白底红字错误显示
- 0x0A=亮绿色成功标识

修改了显示信息



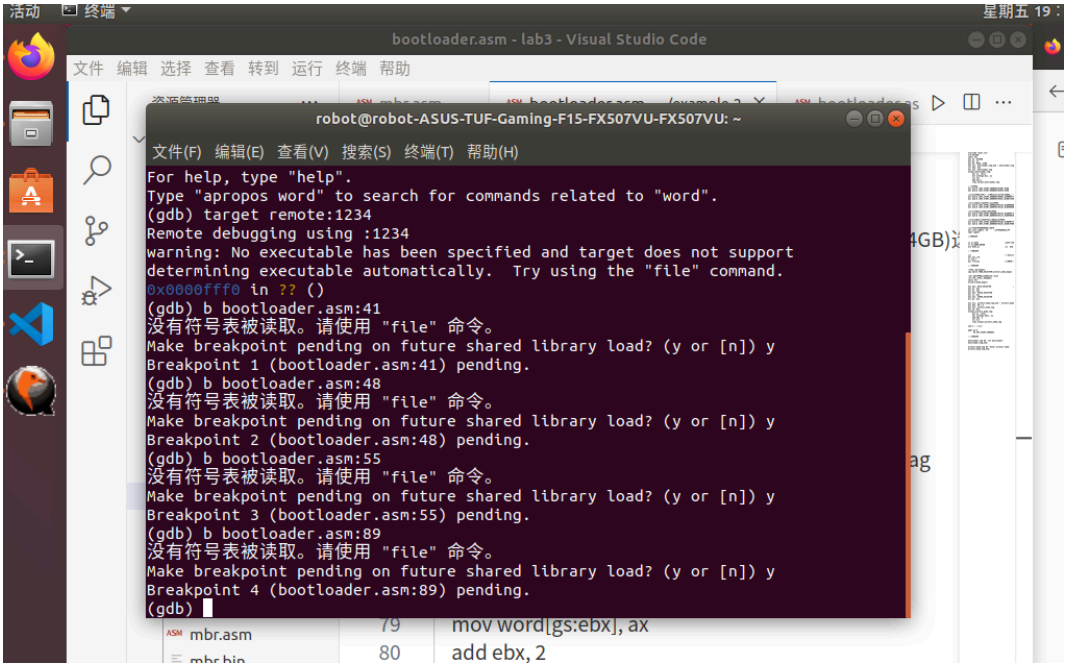
任务二

复现example2

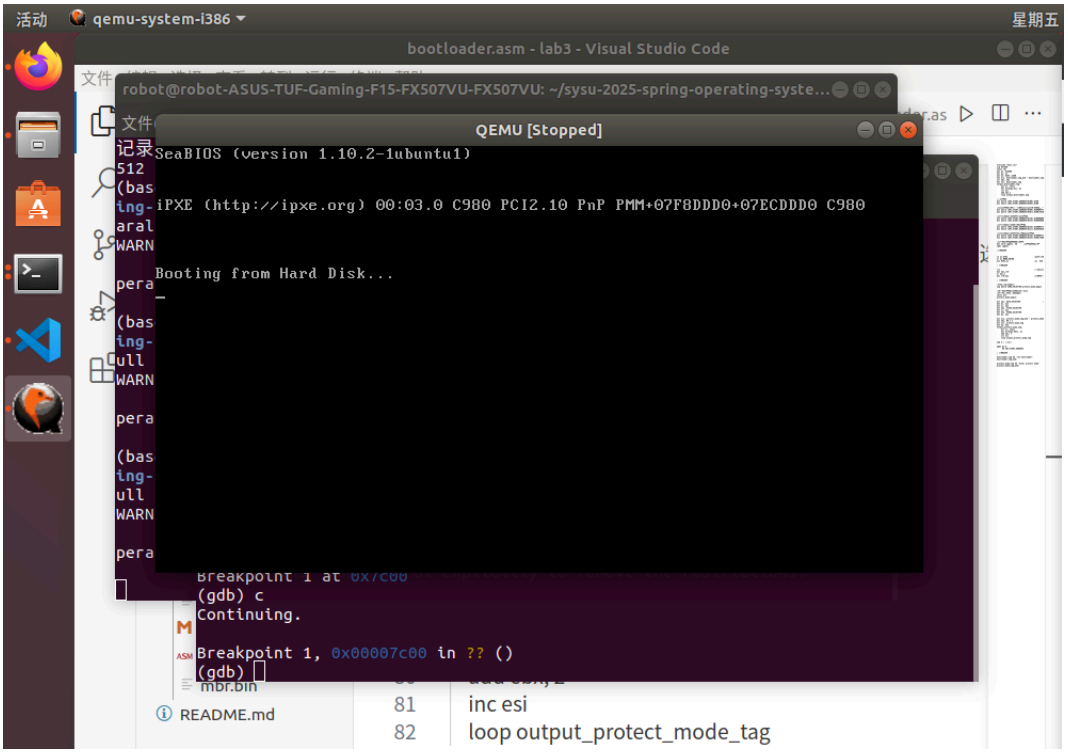


设置断点使用 gdb 进行 debug

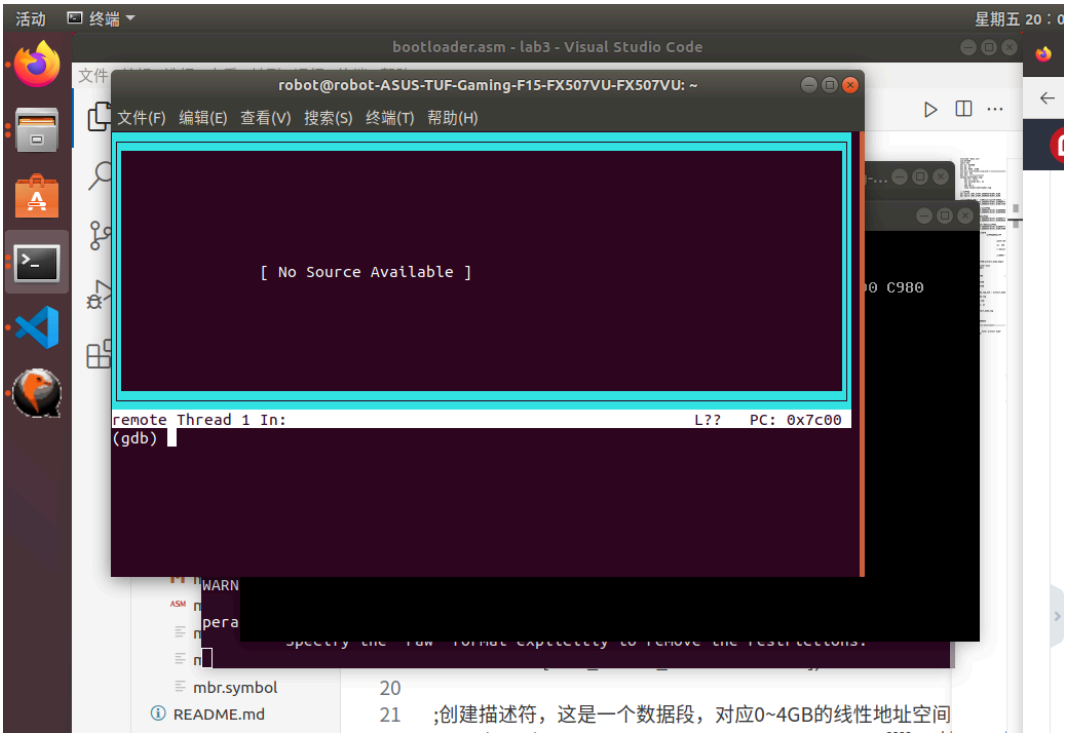
- 在 4 个关键位置设置 4 个断点

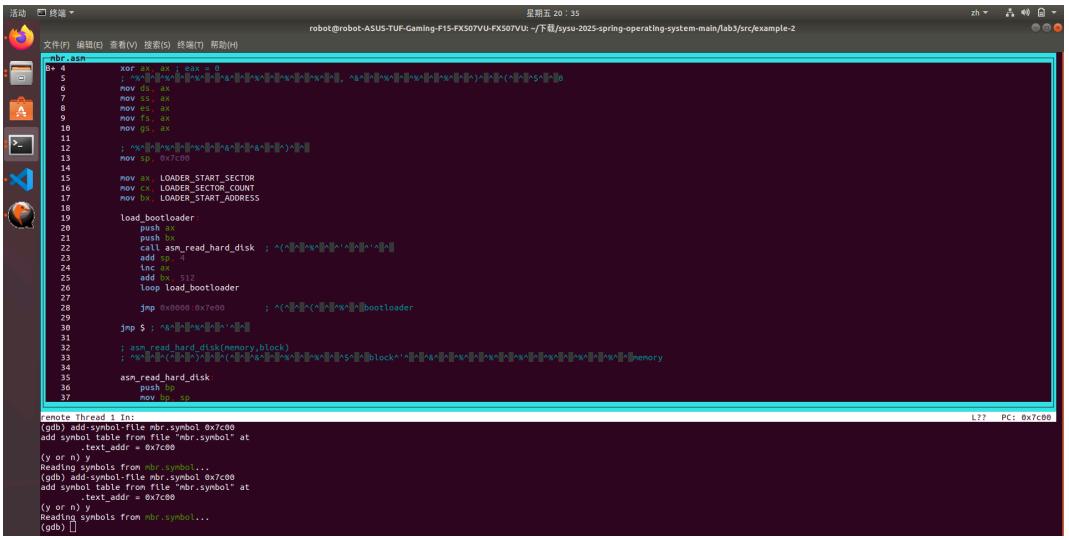

bootloader.asm - lab3 - Visual Studio Code
robot@robot-ASUS-TUF-Gaming-F15-FX507VU-FX507VU: ~
For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) target remote:1234
Remote debugging using :1234
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x0000fff0 in ?? ()
(gdb) b bootloader.asm:41
没有符号表被读取。请使用 "file" 命令。
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 1 (bootloader.asm:41) pending.
(gdb) b bootloader.asm:48
没有符号表被读取。请使用 "file" 命令。
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 2 (bootloader.asm:48) pending.
(gdb) b bootloader.asm:55
没有符号表被读取。请使用 "file" 命令。
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 3 (bootloader.asm:55) pending.
(gdb) b bootloader.asm:89
没有符号表被读取。请使用 "file" 命令。
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 4 (bootloader.asm:89) pending.
(gdb)
mov word[gs:ebx], ax
add ebx, 2

- 断点 1

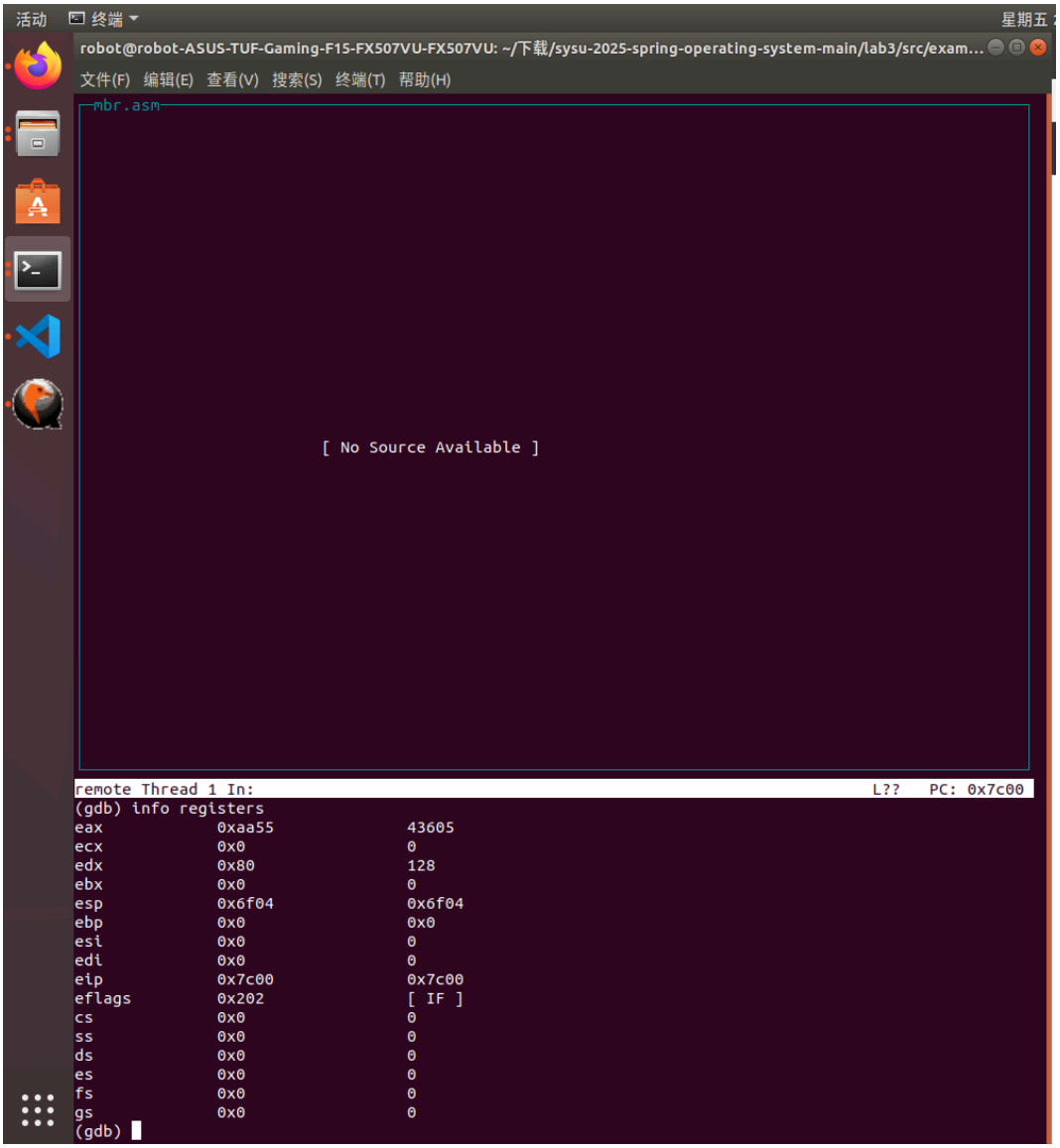

bootloader.asm - lab3 - Visual Studio Code
robot@robot-ASUS-TUF-Gaming-F15-FX507VU-FX507VU: ~/sysu-2025-spring-operating-syste...
QEMU [Stopped]
SeaBIOS (version 1.10.2-1ubuntu1)
iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDD0+07ECDDD0 C980
Booting from Hard Disk...
(gdb) c
Continuing.
Breakpoint 1, 0x00007c00 in ?? ()
(gdb)
inc esi
loop output_protect_mode_tag

- 打开源码显示

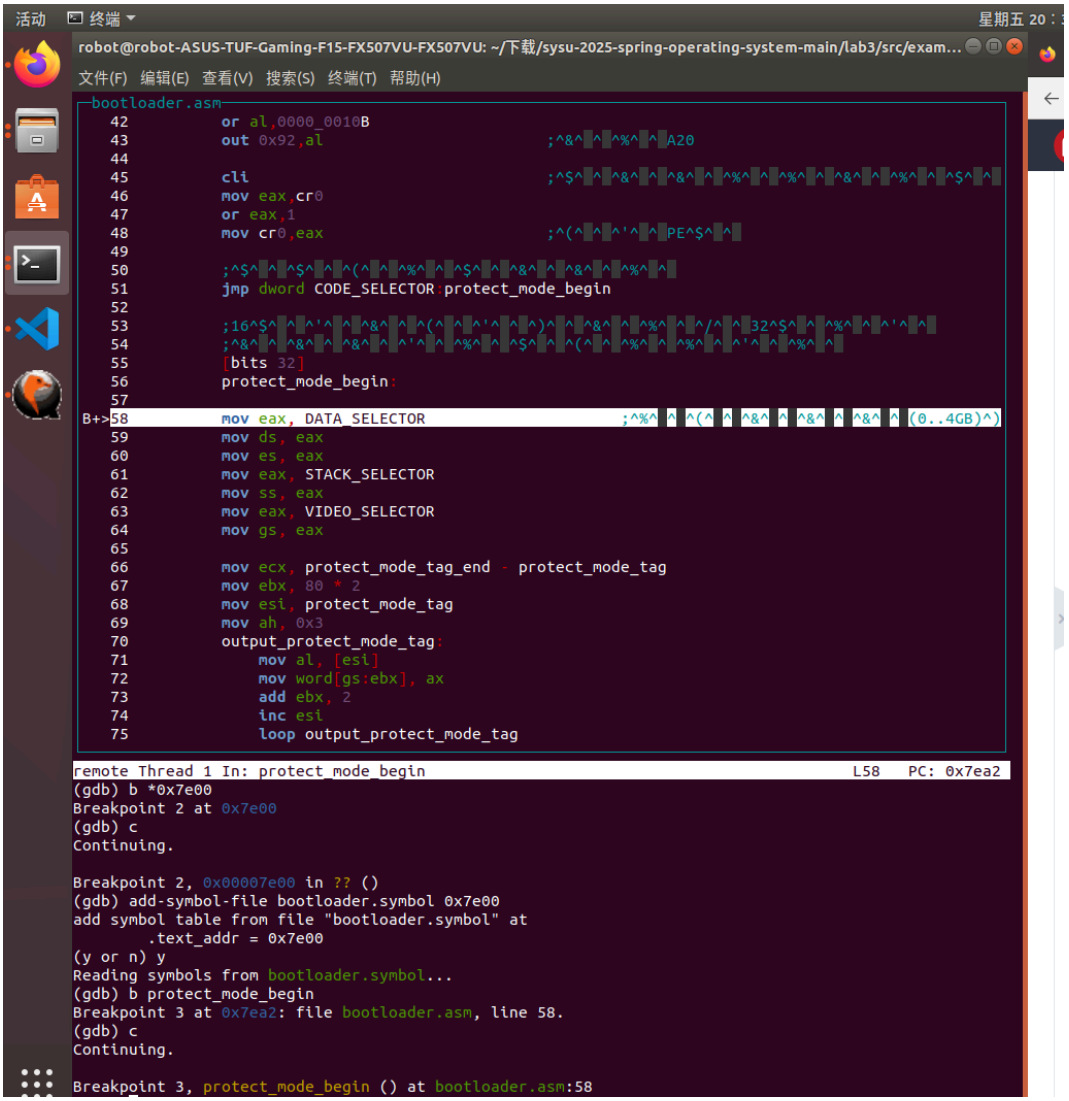




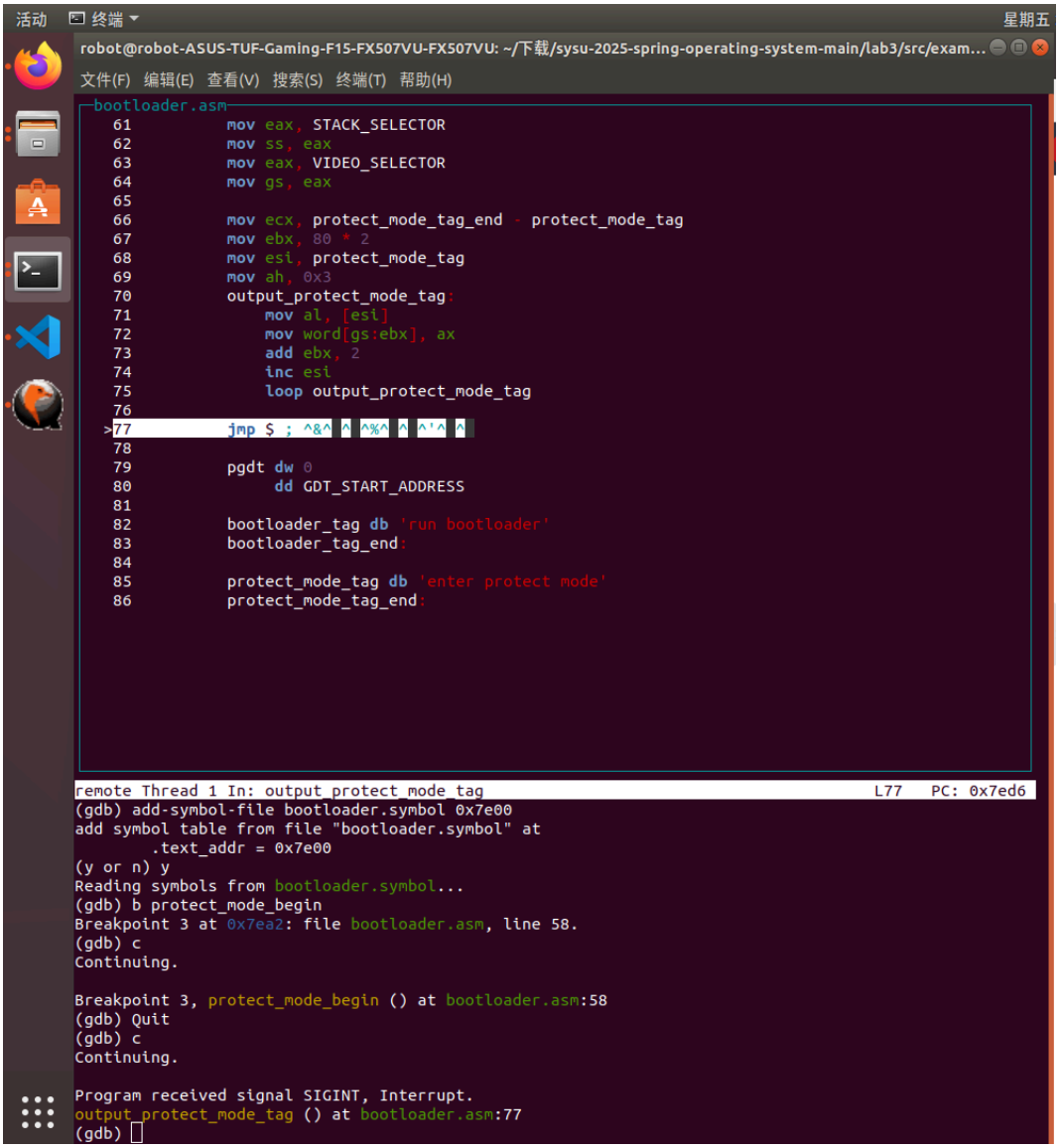
- info registers 检查寄存器内容



- 设置断点b protect_mode_begin


活动
终端
星期五 20
robot@robot-ASUS-TUF-Gaming-F15-FX507VU-FX507VU: ~/下载/sysu-2025-spring-operating-system-main/lab3/src/exam...
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
bootloader.asm
42 or al,0000_0010B
43 out 0x92,al ;^&^ ^ ^%^ ^ A20
44
45 cli ;^$^ ^ ^&^ ^ ^&^ ^ ^%^ ^ ^%^ ^ ^&^ ^ ^%^ ^ ^$^ ^
46 mov eax,cr0
47 or eax,1
48 mov cr0,eax ;^(^ ^ ^'^ ^ PE^$^ ^
49
50 ;^$^ ^ ^$^ ^ ^(^ ^ ^%^ ^ ^$^ ^ ^&^ ^ ^&^ ^ ^%^ ^
51 jmp dword CODE_SELECTOR:protect_mode_begin
52
53 ;16^$^ ^ ^'^ ^ ^&^ ^ ^(^ ^ ^'^ ^ ^)^ ^ ^&^ ^ ^%^ ^ ^/^ ^ 32^$^ ^ ^%^ ^ ^'^ ^
54 ;^&^ ^ ^&^ ^ ^&^ ^ ^'^ ^ ^%^ ^ ^$^ ^ ^(^ ^ ^%^ ^ ^%^ ^ ^'^ ^ ^%^ ^
55 [bits 32]
56 protect_mode_begin:
57
B+>58 mov eax, DATA_SELECTOR ;^%^ ^ ^(^ ^ ^&^ ^ ^&^ ^ ^&^ ^ (0..4GB)^)
59 mov ds, eax
60 mov es, eax
61 mov eax, STACK_SELECTOR
62 mov ss, eax
63 mov eax, VIDEO_SELECTOR
64 mov gs, eax
65
66 mov ecx, protect_mode_tag_end - protect_mode_tag
67 mov ebx, 80 * 2
68 mov esi, protect_mode_tag
69 mov ah, 0x3
70 output_protect_mode_tag:
71 mov al, [esi]
72 mov word[gs:ebx], ax
73 add ebx, 2
74 inc esi
75 loop output_protect_mode_tag
remote Thread 1 In: protect_mode_begin L58 PC: 0x7ea2
(gdb) b *0x7e00
Breakpoint 2 at 0x7e00
(gdb) c
Continuing.
Breakpoint 2, 0x00007e00 in ?? ()
(gdb) add-symbol-file bootloader.symbol 0x7e00
add symbol table from file "bootloader.symbol" at
.text_addr = 0x7e00
(y or n) y
Reading symbols from bootloader.symbol...
(gdb) b protect_mode_begin
Breakpoint 3 at 0x7ea2: file bootloader.asm, line 58.
(gdb) c
Continuing.
Breakpoint 3, protect_mode_begin () at bootloader.asm:58

- 在 jmp 处中断


活动
终端
星期五
robot@robot-ASUS-TUF-Gaming-F15-FX507VU-FX507VU: ~/下载/sysu-2025-spring-operating-system-main/lab3/src/exam...
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
bootloader.asm
61 mov eax, STACK_SELECTOR
62 mov ss, eax
63 mov eax, VIDEO_SELECTOR
64 mov gs, eax
65
66 mov ecx, protect_mode_tag_end - protect_mode_tag
67 mov ebx, 80 * 2
68 mov esi, protect_mode_tag
69 mov ah, 0x3
70 output_protect_mode_tag:
71 mov al, [esi]
72 mov word[gs:ebx], ax
73 add ebx, 2
74 inc esi
75 loop output_protect_mode_tag
76
>77 jmp $; ^&^ ^ ^%^ ^ ^'^ ^
78
79 pgdt dw 0
80 dd GDT_START_ADDRESS
81
82 bootloader_tag db 'run bootloader'
83 bootloader_tag_end:
84
85 protect_mode_tag db 'enter protect mode'
86 protect_mode_tag_end:
remote Thread 1 In: output_protect_mode_tag L77 PC: 0x7ed6
(gdb) add-symbol-file bootloader.symbol 0x7e00
add symbol table from file "bootloader.symbol" at
.text_addr = 0x7e00
(y or n) y
Reading symbols from bootloader.symbol...
(gdb) b protect_mode_begin
Breakpoint 3 at 0x7ea2: file bootloader.asm, line 58.
(gdb) c
Continuing.
Breakpoint 3, protect_mode_begin () at bootloader.asm:58
(gdb) Quit
(gdb) c
Continuing.
Program received signal SIGINT, Interrupt.
output_protect_mode_tag () at bootloader.asm:77
(gdb)

- 显示 gdt 内容

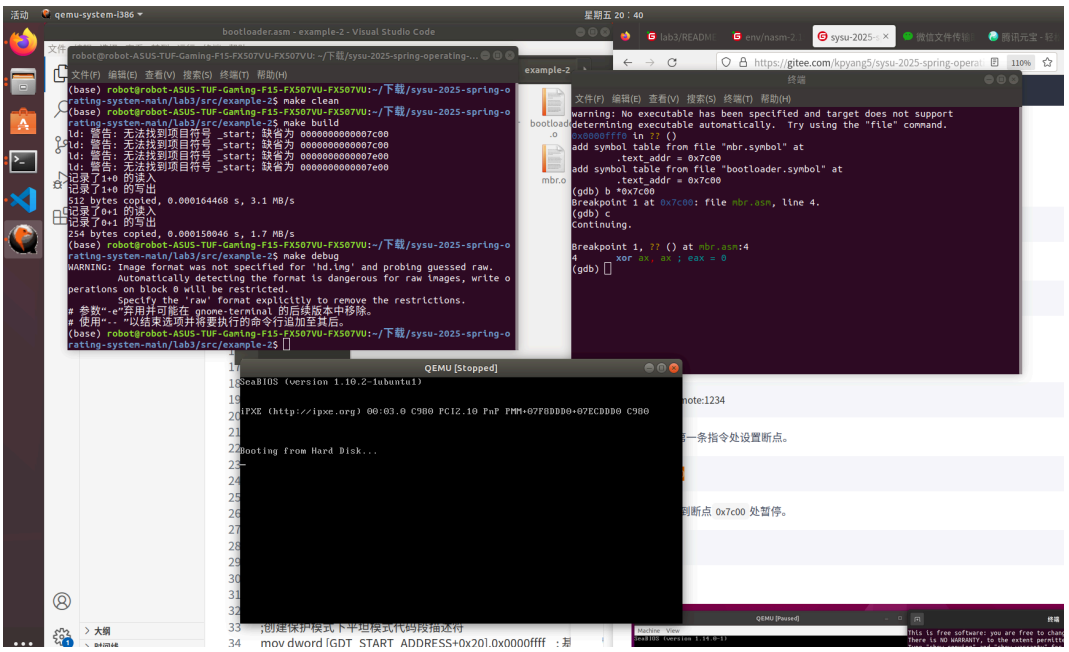
- 过程说明

1. **准备GDT并加载GDTR**

   - 代码在 `GDT_START_ADDRESS` 处填充GDT表项：
     - 写入空描述符。
     - 写入覆盖0-4GB的数据段描述符（`DATA_SELECTOR`）。
     - 写入堆栈段描述符（`STACK_SELECTOR`）。
     - 写入指向显存区域的段描述符（`VIDEO_SELECTOR`）。
     - 写入覆盖0-4GB的代码段描述符（`CODE_SELECTOR`）。
   - `pgdt` 内存位置被设置为GDT的界限（39）和基地址（`GDT_START_ADDRESS`）。
   - `lgdt [pgdt]` 指令将 `pgdt` 中的界限和基地址加载到GDTR寄存器。

2. **打开A20地址线**
   - `in al, 0x92` 读取端口0x92状态。
   - `or al, 0000_0010B` 设置 `al` 中的A20使能位。
   - `out 0x92, al` 将修改后的值写回端口0x92，启用A20 Gate，允许访问超过1MB内存。
3. **设置CR0保护模式位**
   - `mov eax, cr0` 读取 `CR0` 寄存器值。
   - `or eax, 1` 将 `eax` 中的PE（Protection Enable）位置1。
   - `mov cr0, eax` 将更新后的值写回 `CR0`，CPU进入保护模式，分段机制依据GDT工作。
4. **远跳转进入保护模式执行**
   - `jmp dword CODE_SELECTOR:protect_mode_begin` 执行远跳转。
   - 该指令清空CPU指令流水线，将 `CODE_SELECTOR` (0x20) 加载到 `CS` 寄存器，并根据GDT缓存代码段描述符。
   - `EIP` 被设置为 `protect_mode_begin` 的地址。
   - CPU开始在 `protect_mode_begin` 处执行32位保护模式代码。

使用make的debug



3. 关键代码

CHS模式读取磁盘

```
[bits 16]
org 0x7c00

xor ax, ax ; eax = 0
; 初始化段寄存器
mov ds, ax
mov ss, ax
mov es, ax
mov fs, ax
mov gs, ax

; 初始化栈指针
mov sp, 0x7c00

; 显示MBR加载中的信息
mov ax, 0xb800
mov gs, ax
mov ah, 0x02 ; 绿色
mov al, 'M'
mov [gs:0], ax
mov al, 'B'
mov [gs:2], ax
mov al, 'R'
mov [gs:4], ax

; 显示1号扇区内容 - 模拟CHS读取
mov ax, 1    ; 逻辑扇区号为1
call read_sector_chs

; 直接显示bootloader消息
mov ax, 0xb800
```

```
mov gs, ax
mov ah, 0x4F ; 白底红字
mov ecx, bootloader_msg_end - bootloader_msg
mov ebx, 80*2 ; 在第二行开始显示，更明显
mov esi, bootloader_msg
output_bootloader_msg:
    mov al, [esi]
    mov word[gs:bx], ax
    inc esi
    add ebx,2
    loop output_bootloader_msg

jmp $ ; 死循环

; CHS方式读取硬盘函数
; 参数: ax=逻辑扇区号
read_sector_chs:
    push ax
    push bx
    push cx
    push dx

    ; LBA到CHS的转换
    ; 计算扇区号: S = (LBA % 63) + 1
    mov dx, 0
    mov cx, 63
    div cx        ; ax = LBA / 63, dx = LBA % 63
    inc dx        ; S = (LBA % 63) + 1
    mov cl, dl    ; CL低6位存储扇区号

    ; 计算磁头号和柱面号
    mov dx, 0
    mov cx, 18
    div cx        ; ax = (LBA / 63) / 18, dx = (LBA / 63) % 18

    ; DH = 磁头号 = (LBA / 63) % 18
    mov dh, dl
```

```
; 柱面号 = (LBA / 63) / 18
mov ch, al    ; CH = 柱面号低8位

; 柱面号高2位放入CL的高2位
shl ah, 6
or cl, ah

mov dl, 0x80   ; 驱动器号 = 80h (第一个硬盘)
mov ax, 0x0201 ; AH=02h(读扇区), AL=01(读1个扇区)

; ES:BX = 目标地址
xor ax, ax
mov es, ax
mov bx, 0x8000 ; 随便一个地址，用于测试

mov ax, 0x0201 ; 恢复读扇区命令

int 0x13      ; 调用BIOS中断读取扇区
jc .error     ; 如果出错(CF=1)，跳转到错误处理

; 显示CHS成功读取标记
mov ax, 0xb800
mov gs, ax
mov ah, 0x0A ; 亮绿色
mov al, 'C'
mov [gs:10], ax
mov al, 'H'
mov [gs:12], ax
mov al, 'S'
mov [gs:14], ax
mov al, '!'
mov [gs:16], ax

pop dx
pop cx
pop bx
pop ax
ret
```

```
.error:
    ; 显示错误信息
    mov cx, ax    ; 保存错误代码

    mov ax, 0xb800
    mov gs, ax
    mov ah, 0x4F  ; 白底红字
    mov al, 'E'
    mov [gs:40], ax
    mov al, 'R'
    mov [gs:42], ax
    mov al, 'R'
    mov [gs:44], ax

    ; 显示AH中的错误代码
    mov al, ch
    mov [gs:46], ax

    pop dx
    pop cx
    pop bx
    pop ax
    ret

bootloader_msg db 'CHS mode success'
bootloader_msg_end:

times 510 - ($ - $$) db 0
db 0x55, 0xaa
```

4. 实验结果

实验结果同第二部分，已经展示，此处不重复

5. 总结

一、实验核心收获

本实验通过实现从实模式到保护模式的切换，揭示了操作系统启动过程中最关键的硬件控制与内存管理机制。主要收获体现在：

1. **硬件交互能力**：掌握了通过I/O端口直接操作硬盘控制器（LBA模式）的技术细节，理解端口0x1F0-0x1F7的功能分配与状态位检测机制（如忙等待的判断条件 `0x88 & 0x08`）。
2. **保护模式核心机制**：构建全局描述符表（GDT）时，通过平坦模式（Base=0, Limit=4GB）实现零成本地址转换，验证了 `lgdt` 指令与CR0寄存器PE位设置的关键作用。
3. **调试技能提升**：运用GDB的断点设置（`b *0x7e00`）、寄存器查看（`info registers`）和内存检查（`x/10xw 0x8800`），成功跟踪了GDTR加载、A20地址线启用等关键步骤。

二、技术难点与解决方案

1. **LBA模式与CHS转换**：在任务1.2中，通过公式 `柱面 = LBA/(磁头数*每磁道扇区数)` 实现逻辑扇区到物理位置的映射，验证了BIOS中断 `int 13h` 与直接端口操作的本质差异。
2. **段描述符构建**：显存段描述符的特殊处理（基地址0xB8000，界限0x7FFF）需与实模式显存访问对比，通过GDB内存检查确认 `gs:ebx` 的写入有效性。
3. **保护模式验证**：在QEMU中观察到CS寄存器从实模式的 `0x0000` 变为保护模式的 `0x0008`（代码段选择子），并通过 `mov eax, cr0` 指令确认PE位成功置1。

6.参考

LBA与C/H/S相互转化计算--计算扇区数量,含c++计算_lba总扇区数算法-CSDN博客

文章浏览阅读3.9k次。计算扇区时需要转换 LBA与C/H/SC/H/S | Cylinder/Head/Sector | 柱面/磁头/扇区，是旧式磁盘寻道方式LBA | Logical Block Address ,是抽象的扇区，可以理解为线性直铺的扇区，从0–maxCS表示起始柱面号，HS表示起始磁头号，SS表示起始扇区号，PS表示每磁道扇区数，PH表示每柱面磁

https://blog.csdn.net/c5113620/article/details/84112888?ops_request_misc=%257B%2522request%255Fid%2522%253A%252244b9975c6a689188d04b23c5a399e6ea%2522%252C%2522scm%2522%253A%252220140713.130102334..%2522%257D&request_id=44b9975c6a689188d04b23c5a399e6ea&biz_id=0&utm_medium=distribute.pc_search_result.none-task-blog-2~all~sobaiduend~default-2-84112888-null-null.142%5Ev102%5Epc_search_result_base4&utm_term=C%2FH%2FS%E4%B8%8ELBA%E7%9A%84%E8%BD%AC%E6%8D%A2%E5%85%B3%E7%B3%BB&spm=1018.2226.3001.4187